```
================================================================================
Tensor Parallelism + Sequence Parallelism - Backward Gradient Communication
================================================================================

This document explains how gradients are communicated in TP+SP (Tensor Parallelism
combined with Sequence Parallelism) during backward propagation.

Key difference from Vanilla TP: Uses REDUCE-SCATTER instead of ALL-REDUCE!

================================================================================
SETUP: Forward Pass Recap (from sp_tp_example_v0.txt)
================================================================================

Dimensions:
  - Batch (b) = 1, Sequence (s) = 4, Hidden (h) = 4
  - 2 GPUs (GPU0 and GPU1)

SP region: Each GPU has (b, s/2, h) - split along sequence
TP region: Each GPU has (b, s, h) but split hidden - needs all-gather first

Forward pass flow:
  X* (SP) → LayerNorm → g (all-gather) → Column-Linear → GeLU → Row-Linear → g* (reduce-
scatter) → Dropout → V* (SP)

================================================================================
SECTION 1: FORWARD PASS VALUES RECAP
================================================================================

SP Input (SPLIT along sequence):
  X1* (GPU0): tokens 0-1, shape (1,2,4)
  X2* (GPU1): tokens 2-3, shape (1,2,4)

After LayerNorm (still SP):
  Y1* (GPU0): (1,2,4)
  Y2* (GPU1): (1,2,4)

After ALL-GATHER "g" (transition to TP):
  Y (both GPUs): (1,4,4) - full sequence, replicated

After Column-Linear + GeLU (TP region):
  Z1* (GPU0): (1,4,2) - columns 0-1
  Z2* (GPU1): (1,4,2) - columns 2-3

After Row-Linear (partial results):
  W1 (GPU0): (1,4,4) - PARTIAL, needs sum
  W2 (GPU1): (1,4,4) - PARTIAL, needs sum

After REDUCE-SCATTER "g*" (transition back to SP):
  W1* (GPU0): (1,2,4) - tokens 0-1 of (W1+W2)
  W2* (GPU1): (1,2,4) - tokens 2-3 of (W1+W2)

After Dropout (SP):
  V1* (GPU0): (1,2,4)
  V2* (GPU1): (1,2,4)
```

```
================================================================================
SECTION 2: BACKWARD PASS OVERVIEW
================================================================================

Backward flows in REVERSE order:

  dL/dV* → Dropout' → dL/dW* → g (all-gather) → Row-Linear' → dL/dZ*
        → GeLU' → dL/dH* → Column-Linear' → dL/dY → g* (reduce-scatter)
        → LayerNorm' → dL/dX*

┌──────────────────────────────────────────────────────────────────────────┐
│ KEY INSIGHT: BACKWARD IS THE REVERSE OF FORWARD                          │
│                                                                          │
│ Forward "g"  (all-gather):     SP → TP                                   │
│ Backward "g" (reduce-scatter): TP → SP  (conjugate operation!)           │
│                                                                          │
│ Forward "g*" (reduce-scatter): TP → SP                                   │
│ Backward "g*" (all-gather):    SP → TP  (conjugate operation!)           │
└──────────────────────────────────────────────────────────────────────────┘

================================================================================
SECTION 3: DROPOUT BACKWARD (SP Region, No Communication)
================================================================================

Forward: V* = Dropout(W*)
Backward: dL/dW* = dL/dV* ⊙ dropout_mask

Given upstream gradient (SPLIT along sequence):
  dL/dV1* (GPU0): shape (1,2,4) - gradient for tokens 0-1
  dL/dV2* (GPU1): shape (1,2,4) - gradient for tokens 2-3

After dropout backward:
  dL/dW1* (GPU0): (1,2,4)
  dL/dW2* (GPU1): (1,2,4)

NO COMMUNICATION - each GPU processes its sequence chunk independently.

================================================================================
SECTION 4: "g*" BACKWARD = ALL-GATHER (SP → TP Transition)
================================================================================

Forward "g*" was REDUCE-SCATTER: W1,W2 (partial) → W1*,W2* (split)
Backward "g*" is ALL-GATHER: dL/dW1*,dL/dW2* (split) → dL/dW (replicated)

┌──────────────────────────────────────────────────────────────────────────┐
│ MATHEMATICAL JUSTIFICATION                                               │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│ Forward: W* = ReduceScatter(W1, W2)                                      │
│   W1* = (W1 + W2)[tokens 0-1]    (on GPU0)                               │
│   W2* = (W1 + W2)[tokens 2-3]    (on GPU1)                               │
│                                                                          │
│ For gradient, we need dL/dW1 and dL/dW2 given dL/dW1* and dL/dW2*         │
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  Chain rule for W1:                                                      │
│    dL/dW1[tokens 0-1] = dL/dW1* (since W1* depends on W1[0-1])           │
│    dL/dW1[tokens 2-3] = dL/dW2* (since W2* depends on W1[2-3])           │
│    → dL/dW1 = Concat(dL/dW1*, dL/dW2*) = AllGather(dL/dW*)               │
│                                                                          │
│  Same for W2:                                                            │
│    dL/dW2 = AllGather(dL/dW*) = same as dL/dW1                           │
│                                                                          │
│  RESULT: dL/dW1 = dL/dW2 = AllGather([dL/dW1*, dL/dW2*])                 │
└──────────────────────────────────────────────────────────────────────────┘
```

```
Numerical Example:
-----------------
Before all-gather (SP, split):
  dL/dW1* (GPU0): [[[1, 2, 3, 4], [5, 6, 7, 8]]]           (tokens 0-1)
  dL/dW2* (GPU1): [[[9, 10, 11, 12], [13, 14, 15, 16]]]    (tokens 2-3)

After ALL-GATHER (TP, replicated):
  dL/dW (both GPUs): [[[1, 2, 3, 4],      ← token 0 (from dL/dW1*)
                      [5, 6, 7, 8],       ← token 1 (from dL/dW1*)
                      [9, 10, 11, 12],    ← token 2 (from dL/dW2*)
                      [13, 14, 15, 16]]] ← token 3 (from dL/dW2*)
  Shape: (1, 4, 4)

================================================================================
SECTION 5: ROW-LINEAR BACKWARD (TP Region, No Communication)
================================================================================

Forward was: W = Z × B  where Z = [Z1* | Z2*], B = [B1; B2]
             W1 = Z1* × B1 (GPU0 partial)
             W2 = Z2* × B2 (GPU1 partial)

Given: dL/dW (same on both GPUs after all-gather from Section 4)
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│ ROW-LINEAR BACKWARD FORMULAS                                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  Weight gradient:  dL/dB = Z*^T × dL/dW                                  │
│  Input gradient:   dL/dZ* = dL/dW × B^T                                  │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

```
Weight gradient (no communication):
  GPU0: dL/dB1 = Z1*^T × dL/dW    Shape: (2, 4)
  GPU1: dL/dB2 = Z2*^T × dL/dW    Shape: (2, 4)

Input gradient (no communication):
Since B^T = [B1^T | B2^T]:
  dL/dZ = dL/dW × B^T = [dL/dW × B1^T | dL/dW × B2^T]

Each GPU computes its portion:
  GPU0: dL/dZ1* = dL/dW × B1^T    Shape: (1, 4, 2)
```

```
  GPU1: dL/dZ2* = dL/dW × B2^T    Shape: (1, 4, 2)

NO COMMUNICATION - each GPU has dL/dW (from all-gather) and its local B.

Numerical Example:
-----------------
dL/dW (both GPUs):
  [[[1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]]]

B1^T (GPU0):              B2^T (GPU1):
  [[0.1, 0.5],              [[0.5, 0.9],
   [0.2, 0.6],               [0.6, 1.0],
   [0.3, 0.7],               [0.7, 1.1],
   [0.4, 0.8]]               [0.8, 1.2]]

GPU0: dL/dZ1* = dL/dW × B1^T
  Token 0: [1,2,3,4] × B1^T = [1×0.1+2×0.2+3×0.3+4×0.4, 1×0.5+2×0.6+3×0.7+4×0.8]
                            = [3.0, 7.0]
  Token 1: [5,6,7,8] × B1^T = [5×0.1+6×0.2+7×0.3+8×0.4, 5×0.5+6×0.6+7×0.7+8×0.8]
                            = [7.0, 17.0]
  (similarly for tokens 2,3)

dL/dZ1* (GPU0): [[[3.0, 7.0], [7.0, 17.0], [11.0, 27.0], [15.0, 37.0]]]
dL/dZ2* (GPU1): [[[7.0, 11.0], [17.0, 27.0], [27.0, 43.0], [37.0, 59.0]]]

================================================================================
SECTION 6: GELU BACKWARD (No Communication)
================================================================================

Forward: Z* = GeLU(H*)
Backward: dL/dH* = dL/dZ* ⊙ GeLU'(H*)

Each GPU applies element-wise:
  GPU0: dL/dH1* = dL/dZ1* ⊙ GeLU'(H1*)    Shape: (1, 4, 2)
  GPU1: dL/dH2* = dL/dZ2* ⊙ GeLU'(H2*)    Shape: (1, 4, 2)

NO COMMUNICATION.

================================================================================
SECTION 7: COLUMN-LINEAR BACKWARD (TP Region)
================================================================================

Forward was: H = Y × A  where A = [A1 | A2]
             H1* = Y × A1 (GPU0, columns 0-1)
             H2* = Y × A2 (GPU1, columns 2-3)

Given: dL/dH1* on GPU0, dL/dH2* on GPU1

┌──────────────────────────────────────────────────────────────────────────────┐
│ COLUMN-LINEAR BACKWARD FORMULAS                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  Weight gradient:  dL/dA = Y^T × dL/dH                                   │
│  Input gradient:   dL/dY = dL/dH × A^T                                   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

Weight gradient (no communication):
  GPU0: dL/dA1 = Y^T × dL/dH1*    Shape: (4, 2)
  GPU1: dL/dA2 = Y^T × dL/dH2*    Shape: (4, 2)

Input gradient (PARTIAL - needs reduction):
  dL/dY = dL/dH × A^T
        = [dL/dH1* | dL/dH2*] × [A1^T]
                                [A2^T]
        = dL/dH1* × A1^T + dL/dH2* × A2^T

Each GPU computes PARTIAL gradient:
  GPU0: dL/dY_partial = dL/dH1* × A1^T    Shape: (1, 4, 4)
  GPU1: dL/dY_partial = dL/dH2* × A2^T    Shape: (1, 4, 4)

Full gradient: dL/dY = dL/dY_partial(GPU0) + dL/dY_partial(GPU1)

================================================================================
SECTION 8: "g" BACKWARD = REDUCE-SCATTER (TP → SP Transition)
================================================================================

Forward "g" was ALL-GATHER: Y1*,Y2* (split) → Y (replicated)
Backward "g" is REDUCE-SCATTER: dL/dY_partial → dL/dY1*,dL/dY2* (split)

┌──────────────────────────────────────────────────────────────────────────┐
│  THIS IS THE KEY DIFFERENCE FROM VANILLA TP!                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  VANILLA TP: dL/dX is REPLICATED → needs ALL-REDUCE                      │
│                                                                          │
│  SP + TP: dL/dY* is SPLIT → can use REDUCE-SCATTER                       │
│           (50% less communication!)                                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

Mathematical Justification:
---------------------------
Forward: Y = AllGather([Y1*, Y2*])
         Y = Concat(Y1*, Y2*, dim=sequence)

For gradient:
  dL/dY1* = dL/dY[tokens 0-1]  (gradient flows to Y1* from tokens 0-1)
  dL/dY2* = dL/dY[tokens 2-3]  (gradient flows to Y2* from tokens 2-3)

But dL/dY = dL/dY_partial(GPU0) + dL/dY_partial(GPU1) (sum of partials)

Combined operation:
  1. REDUCE: Sum partials across GPUs
  2. SCATTER: Distribute to respective sequence chunks
```

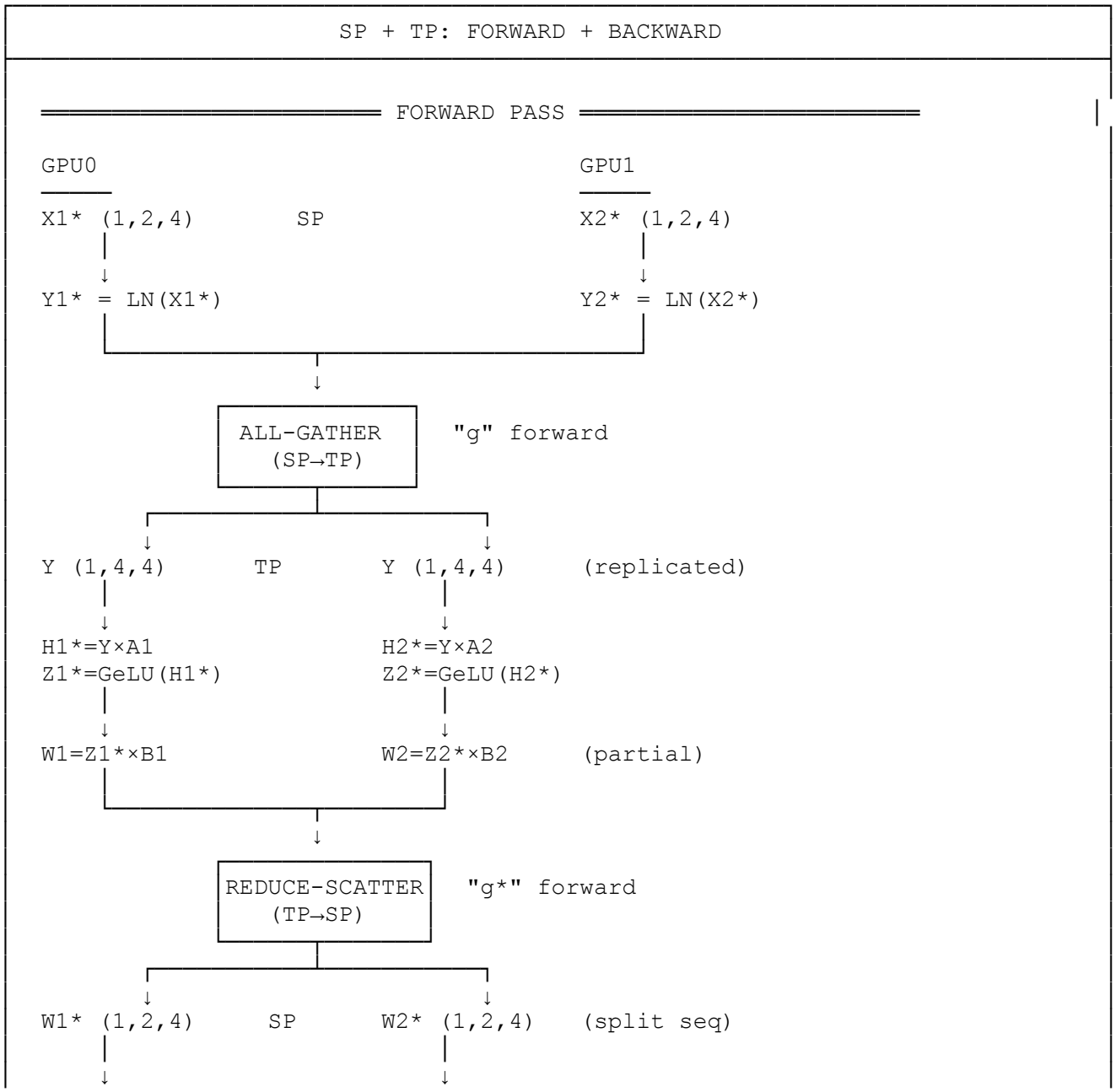
This is exactly REDUCE-SCATTER!

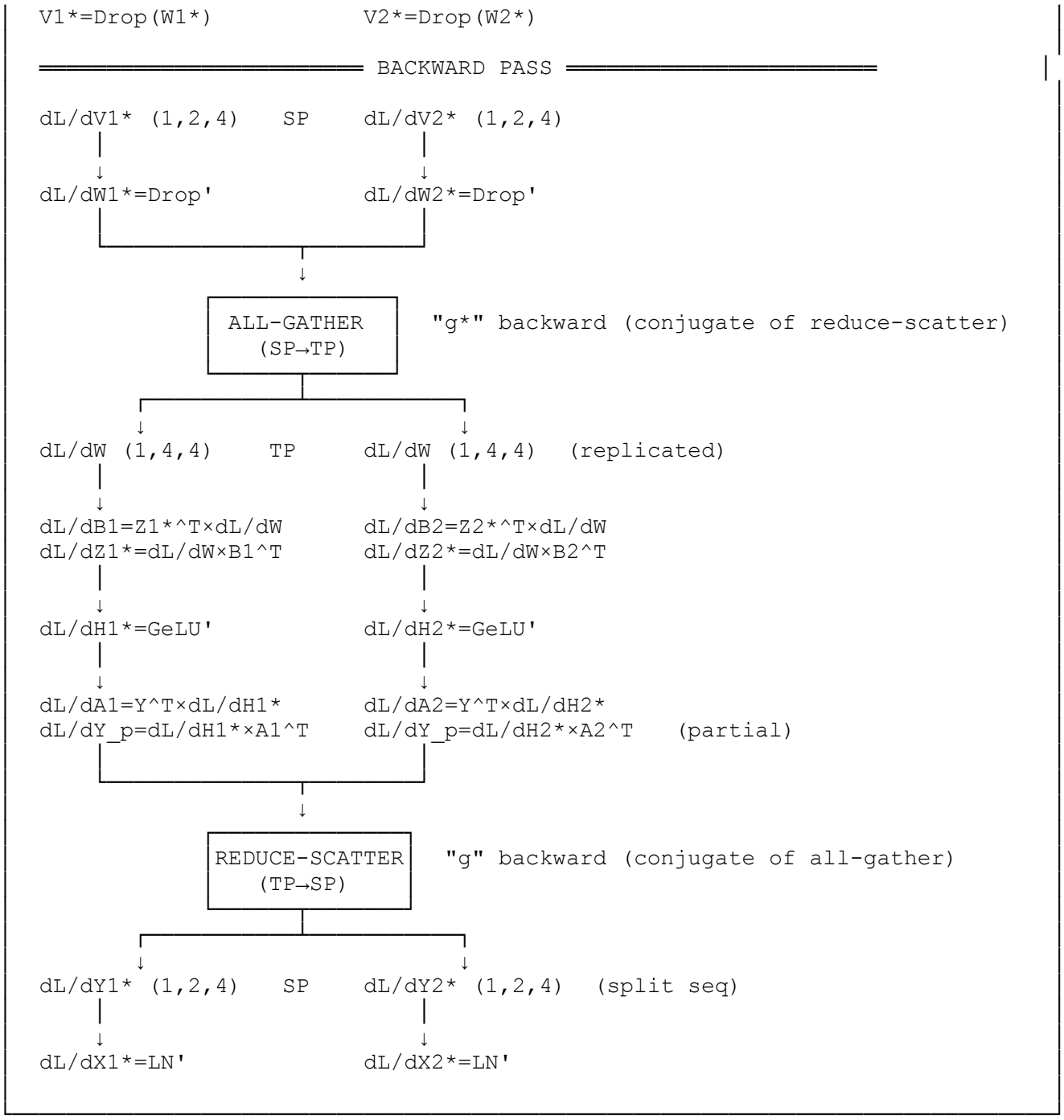
Numerical Example:
```
-----------------
GPU0 partial: dL/dY_partial                GPU1 partial: dL/dY_partial
  [[[0.5, 1.7, 2.9, 4.1],                    [[[2.5, 5.3, 8.1, 10.9],
    [1.7, 6.1, 10.5, 14.9],                    [5.3, 11.3, 17.3, 23.3],
    [2.9, 10.5, 18.1, 25.7],                   [8.1, 17.3, 26.5, 35.7],
    [4.1, 14.9, 25.7, 36.5]]]                  [10.9, 23.3, 35.7, 48.1]]]

After REDUCE-SCATTER:

Step 1: REDUCE (conceptual full sum):
  dL/dY_full = partial0 + partial1
  [[[3.0, 7.0, 11.0, 15.0],      ← Token 0
    [7.0, 17.4, 27.8, 38.2],     ← Token 1
    [11.0, 27.8, 44.6, 61.4],    ← Token 2
    [15.0, 38.2, 61.4, 84.6]]]   ← Token 3

Step 2: SCATTER (distribute):
  GPU0 gets: dL/dY1* = tokens 0-1 = [[[3.0, 7.0, 11.0, 15.0],
                                     [7.0, 17.4, 27.8, 38.2]]]
  GPU1 gets: dL/dY2* = tokens 2-3 = [[[11.0, 27.8, 44.6, 61.4],
                                     [15.0, 38.2, 61.4, 84.6]]]

Shape after: (1, 2, 4) on each GPU - back to SP!

================================================================================
SECTION 9: LAYERNORM BACKWARD (SP Region, No Communication)
================================================================================

Forward: Y* = LayerNorm(X*)
Backward: dL/dX* = LayerNorm_backward(dL/dY*, X*)

Each GPU computes independently:
  GPU0: dL/dX1* = LayerNorm'(dL/dY1*, X1*)    Shape: (1, 2, 4)
  GPU1: dL/dX2* = LayerNorm'(dL/dY2*, X2*)    Shape: (1, 2, 4)

NO COMMUNICATION - each GPU has its own sequence chunk.

================================================================================
SECTION 10: COMPLETE BACKWARD COMMUNICATION PATTERN
================================================================================

┌──────────────────────────────────────────────────────────────────────────────┐
│ SP + TP: BACKWARD PASS COMMUNICATION                                         │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│   Layer              Region    Backward Communication                        │
│   ─────────────────  ──────    ──────────────────────                        │
│   Dropout            SP        None                                          │
│   g* backward        SP→TP     ALL-GATHER (reconstruct dL/dW)                │
│   Row-Linear         TP        None (weight grad local, input grad local)    │
```

GeLU	TP	None
Column-Linear	TP	None (weight grad local, input grad partial)
g backward	TP→SP	REDUCE-SCATTER (sum + distribute dL/dY)
LayerNorm	SP	None

TOTAL: 1 ALL-GATHER + 1 REDUCE-SCATTER

SECTION 11: DIAGRAM - SP+TP FULL FORWARD + BACKWARD





==
SECTION 12: COMMUNICATION COMPARISON - VANILLA TP vs SP+TP
==

VANILLA TP

Forward: Row-Linear output → ALL-REDUCE (Y = Y1 + Y2)

```
Backward:  Column-Linear dL/dX   → ALL-REDUCE (dL/dX = sum of partials)

Communication per MLP:
  Forward:  1 × ALL-REDUCE = 2D × (P-1)/P  (where D = b × s × h)
  Backward: 1 × ALL-REDUCE = 2D × (P-1)/P
  Total:    4D × (P-1)/P
```

SP + TP

```
Forward:   g (SP→TP)   → ALL-GATHER
           g* (TP→SP)  → REDUCE-SCATTER

Backward:  g* backward → ALL-GATHER
           g backward  → REDUCE-SCATTER

Communication per MLP:
  Forward:  1 × ALL-GATHER + 1 × REDUCE-SCATTER
          = D × (P-1)/P + D × (P-1)/P = 2D × (P-1)/P
  Backward: 1 × ALL-GATHER + 1 × REDUCE-SCATTER
          = D × (P-1)/P + D × (P-1)/P = 2D × (P-1)/P
  Total:    4D × (P-1)/P

WAIT - same total? Let's look closer...
```

Actually, the KEY benefit is MEMORY, not just communication:

MEMORY COMPARISON

```
VANILLA TP:
  - Input X: (b, s, h) per GPU (REPLICATED)
  - Activations stored for backward: (b, s, h) per GPU
  - Memory for activations: O(b × s × h) per GPU

SP + TP:
  - Input X*: (b, s/P, h) per GPU (SPLIT)
  - Activations in SP region: (b, s/P, h) per GPU
  - Memory for SP activations: O(b × s/P × h) per GPU
  - P× REDUCTION in activation memory!

For sequence length s = 4096, P = 8 GPUs:
  Vanilla TP: stores 4096 tokens of activations per GPU
  SP + TP:    stores 512 tokens of activations per GPU (8× less!)
```

```
================================================================================
SECTION 13: DETAILED COMMUNICATION VOLUME ANALYSIS
================================================================================
```

For one MLP layer with dimensions (b, s, h):

Metric	Vanilla TP	SP + TP
Forward Comm		
- Operation 1	ALL-REDUCE size: (b,s,h) bytes: 2×D×(P-1)/P	ALL-GATHER size: (b,s/P,h)→(b,s,h) bytes: D×(P-1)/P
- Operation 2	(none)	REDUCE-SCATTER size: (b,s,h)→(b,s/P,h) bytes: D×(P-1)/P
Forward Total	2D×(P-1)/P	2D×(P-1)/P
Backward Comm		
- Operation 1	ALL-REDUCE size: (b,s,h) bytes: 2×D×(P-1)/P	ALL-GATHER size: (b,s/P,h)→(b,s,h) bytes: D×(P-1)/P
- Operation 2	(none)	REDUCE-SCATTER size: (b,s,h)→(b,s/P,h) bytes: D×(P-1)/P
Backward Total	2D×(P-1)/P	2D×(P-1)/P
TOTAL per MLP (same comm volume)	4D×(P-1)/P	4D×(P-1)/P
MEMORY BENEFIT	activations: O(s)	activations: O(s/P) P× memory reduction!

Key: D = b × s × h (full tensor size)

==
SECTION 14: WHY SP+TP BACKWARD USES REDUCE-SCATTER (NOT ALL-REDUCE)
==

THE CRUCIAL INSIGHT

In Vanilla TP:
  - X is REPLICATED → dL/dX must be REPLICATED → ALL-REDUCE

In SP + TP:
  - Y* is SPLIT along sequence → dL/dY* can be SPLIT → REDUCE-SCATTER

The data distribution DETERMINES the communication pattern!

Proof by chain rule:

Forward all-gather:
  Y[token i] = Y1*[token i] for i ∈ [0, s/2)   (from GPU0)
  Y[token i] = Y2*[token i] for i ∈ [s/2, s)   (from GPU1)

Backward:
  dL/dY1*[token i] = dL/dY[token i] for i ∈ [0, s/2)   → goes to GPU0
  dL/dY2*[token i] = dL/dY[token i] for i ∈ [s/2, s)   → goes to GPU1

Since dL/dY = dL/dY_partial(GPU0) + dL/dY_partial(GPU1):

  dL/dY1* = (dL/dY_partial0 + dL/dY_partial1)[tokens 0 to s/2-1]
  dL/dY2* = (dL/dY_partial0 + dL/dY_partial1)[tokens s/2 to s-1]

This is exactly REDUCE (sum) + SCATTER (distribute) = REDUCE-SCATTER!

==
SECTION 15: SUMMARY - SP+TP BACKWARD GRADIENT COMMUNICATION
==

SP + TP BACKWARD: KEY POINTS

1. Backward is the CONJUGATE of forward:
   - Forward all-gather (SP→TP)   ⟷ Backward reduce-scatter (TP→SP)
   - Forward reduce-scatter (TP→SP) ⟷ Backward all-gather (SP→TP)

2. Communication pattern:
   - "g*" backward: ALL-GATHER (reconstruct full dL/dW from split dL/dW*)
   - "g" backward: REDUCE-SCATTER (sum dL/dY partials, distribute to SP)

3. Why REDUCE-SCATTER (not ALL-REDUCE)?
   - Input Y* is SPLIT along sequence dimension
   - Therefore dL/dY* only needs the SPLIT result
   - No need to replicate → REDUCE-SCATTER is sufficient

4. Main benefit of SP+TP:
   - Communication volume: SAME as vanilla TP
   - Memory for activations: P× REDUCTION (split along sequence)
   - Enables training longer sequences with limited GPU memory

5. Weight gradients:
   - dL/dA1, dL/dA2: computed locally, no communication
   - dL/dB1, dL/dB2: computed locally, no communication
   - Only input gradients require collective communication

==

SECTION 16: FORMULAS SUMMARY
==

SP + TP BACKWARD FORMULAS

Given: dL/dV1* (GPU0), dL/dV2* (GPU1) - gradients for outputs

Dropout backward:
  dL/dW1* = dL/dV1* ⊙ mask1
  dL/dW2* = dL/dV2* ⊙ mask2

"g*" backward (ALL-GATHER):
  dL/dW = AllGather([dL/dW1*, dL/dW2*], dim=sequence)
        = Concat(dL/dW1*, dL/dW2*, dim=1)   // both GPUs get full dL/dW

Row-Linear backward:
  dL/dB1 = Z1*^T × dL/dW   (GPU0, no comm)
  dL/dB2 = Z2*^T × dL/dW   (GPU1, no comm)
  dL/dZ1* = dL/dW × B1^T   (GPU0)
  dL/dZ2* = dL/dW × B2^T   (GPU1)

GeLU backward:
  dL/dH1* = dL/dZ1* ⊙ GeLU'(H1*)
  dL/dH2* = dL/dZ2* ⊙ GeLU'(H2*)

Column-Linear backward:
  dL/dA1 = Y^T × dL/dH1*   (GPU0, no comm)
  dL/dA2 = Y^T × dL/dH2*   (GPU1, no comm)
  dL/dY_partial0 = dL/dH1* × A1^T   (GPU0, partial)
  dL/dY_partial1 = dL/dH2* × A2^T   (GPU1, partial)

"g" backward (REDUCE-SCATTER):
  [dL/dY1*, dL/dY2*] = ReduceScatter([dL/dY_p0, dL/dY_p1], dim=sequence)
  dL/dY1* = (dL/dY_p0 + dL/dY_p1)[tokens 0:s/2]   // GPU0
  dL/dY2* = (dL/dY_p0 + dL/dY_p1)[tokens s/2:s]   // GPU1

LayerNorm backward:
  dL/dX1* = LayerNorm'(dL/dY1*, X1*)   (GPU0, no comm)
  dL/dX2* = LayerNorm'(dL/dY2*, X2*)   (GPU1, no comm)

==
END OF DOCUMENT
==